Capstone Project 4: High-Availability E-Commerce Microservices

Scenario: A global retail brand is migrating its legacy monolithic checkout system to a scalable microservices architecture to handle Black Friday traffic surges.

- **Problem Statement:** The current system crashes during high traffic, lacks real-time inventory updates, and has security vulnerabilities in user session management.
- **Case:** Scale compute independently of the database, ensure sub-millisecond session retrieval, and integrate enterprise-grade SSO.
- **Solution Components:**
  - **Compute:** ECS Fargate with ALB (Auto-scaling).
  - **Cache:** ElastiCache (Redis) for session state.
  - **Data:** RDS Aurora (MySQL) with ElastiCache as a "Read-aside" cache.
  - **Security:** Okta OIDC integration via Application Load Balancer (ALB).

Terraform Snippet (Core Components)

Terraform
```
# ECS Service with ALB and Redis Integration
resource "aws_ecs_service" "app" {
  name            = "checkout-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count   = 3
  launch_type     = "FARGATE"

  load_balancer {
    target_group_arn = aws_lb_target_group.app.arn
    container_name   = "checkout-api"
    container_port   = 8080
  }
}

# ElastiCache Redis Cluster for Session Management
resource "aws_elasticache_cluster" "session_store" {
  cluster_id           = "session-redis"
  engine               = "redis"
  node_type            = "cache.t3.micro"
  num_cache_nodes      = 1
  parameter_group_name = "default.redis7"
  security_group_ids   = [aws_security_group.redis_sg.id]
}
```